

Tema VI – Listas

En el ámbito de las estructuras de datos, las listas representan uno de los pilares fundamentales sobre los que se construyen sistemas de almacenamiento y organización de información más complejos. A diferencia de las estructuras estáticas como los arrays, que requieren dimensionamiento previo y presentan limitaciones en su capacidad de expansión, las listas ofrecen una flexibilidad incomparable al permitir la adición y eliminación dinámica de elementos sin restricciones de tamaño predefinido.

Una lista es una estructura de datos lineal y dinámica que representa una secuencia de elementos, generalmente del mismo tipo. Cada elemento se conecta al siguiente mediante un "enlace" o referencia, formando una cadena de nodos. Esta característica permite tener los elementos en secuencia con la capacidad de llevar a cabo inserciones y eliminaciones eficientes en cualquier posición, a la vez que se hace un uso óptimo de memoria ya que solo se asigna lo necesario.

1. El TAD Lista

El TAD Lista es un ejemplo clásico de abstracción de datos donde se define **qué hace** una lista sin especificar **cómo lo hace**. La flexibilidad de las listas hace que no exista una única definición; más bien, el contrato de operaciones depende del tipo de lista que se desee implementar.

Por su flexibilidad, no existe una única definición del TAD lista: debe determinarse primero el tipo de lista que se desea, lo cual condiciona sus operaciones (su contrato), ya que, por ejemplo, una lista sin restricciones permite operaciones no toleradas en una lista ordenada, tales como `insertar_final`. En todo caso, la especificación en lenguaje natural de una lista general sería:

- **Nombre:** Lista.
- **Valores posibles:** vacía o conteniendo una secuencia de elementos del tipo base.
- **Operaciones:**
 - `insert_at_end(list, elem)`: inserta un elemento al final de la lista
 - `insert_at_begin(list, elem)`: inserta un elemento al principio de la lista
 - `delete_at_end(list)`: elimina el elemento al final de la lista
 - `delete_at_begin(list)`: elimina el elemento al principio de la lista
 - `first(list)`: accede al primer elemento de la lista
 - `last(list)`: accede al último elemento de la lista
 - `is_empty(list)`: comprueba si la lista está vacía
 - `in_list(list, elem)`: determina si un cierto elemento está en la lista
- **Uso de otros tipos:** Depende de los elementos que contenga y de las estructuras auxiliares utilizadas en la implementación. A menudo también, punteros.

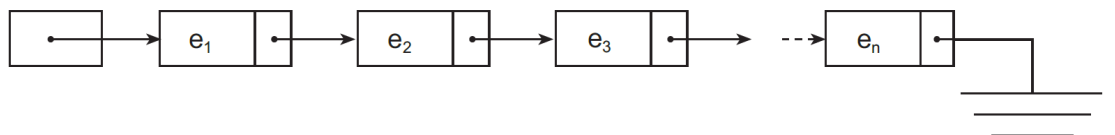


Ilustración 13. Lista enlazada simple

La implementación de listas en estructuras de soporte estáticas como arrays no es habitual, siendo en casi la totalidad de ocasiones preferida una implementación en memoria dinámica. Atendiendo a eso, su implementación puede ser distinta. Así, tendremos listas enlazadas simples, donde en cada nodo hay una referencia o apuntador a otro nodo; listas doblemente enlazadas, donde cada

nodo referencia al anterior y al siguiente; y lista circular, donde el último nodo referencia al primero, etc.

Por otra parte, dependiendo de las restricciones impuestas, podemos también encontrarnos con diferentes tipos de listas:

1. Lista sin restricciones: Permite todas las operaciones básicas
2. Lista ordenada: Mantiene los elementos en un orden específico
3. Lista con acceso restringido

La implementación de listas en memoria dinámica se basa, como ya vimos en las colas y pilas, en el concepto de nodo, una estructura que contiene el dato a almacenar junto con una referencia al siguiente nodo (y al anterior, en listas dobles).

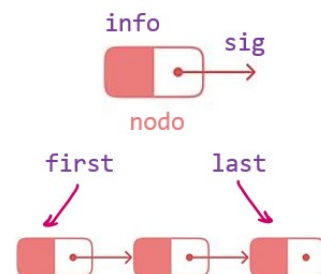
La implementación basada en memoria dinámica permite trabajar solo con el espacio que realmente se necesita, evitando así el desperdicio de tener que asignar un espacio a priori que no se sabe si será adecuado. Además, las operaciones de inserción y eliminación resultan mucho más ágiles, ya que no es necesario desplazar elementos. Otra ventaja importante es que la estructura puede crecer sin límites predeterminados, únicamente condicionada por la memoria disponible en el sistema, a diferencia de lo que ocurre con los arrays estáticos. Esta forma de gestionar los datos aporta una gran flexibilidad, lo que facilita la creación y manejo de estructuras más complejas.

2. Lista enlazada simple

La lista enlazada simple es la forma más básica de lista, donde cada nodo apunta solo al siguiente. En Pascal, su implementación sería la siguiente:

type

```
PNodo = ^tNodo;  
  
tNodo = record  
    info: integer;    // Dato almacenado  
    sig: PNodo;      // Puntero al siguiente nodo  
end;  
  
TListaSimple = record  
    first: PNodo;    // Puntero al primer nodo  
    last: PNodo;    // Puntero al último nodo  
end;
```



Como se ve, la lista es en realidad un registro (record) dos punteros a un nodo de tipo lista (first y last), y esto es así para optimizar las operaciones en ambos extremos de la lista.

Inicialización

Operación para establecer un estado inicial consistente antes de cualquier operación sobre la lista. Tiene complejidad $O(1)$ ya que simplemente asigna valores nulos a dos punteros.

```
procedure initialize(var list: TListaSimple);  
begin  
    list.first := nil;  
    list.last := nil;  
end;
```

Inserciones

La **inserción al final** tiene complejidad $O(1)$ gracias al puntero last. Sin él, sería necesario recorrer toda la lista para encontrar el último nodo, resultando en $O(n)$.

```

procedure insert_at_end(var list: TListaSimple; x: integer);
var
  newNode: PNode;
begin
  new(newNode);           // 1. Reserva memoria para el nuevo nodo
  newNode^.info := x;     // 2. Asigna el valor
  newNode^.sig := nil;    // 3. El nuevo nodo será el último
  if is_empty(list) then // 4. Caso especial: lista vacía
    list.first := newNode
  else                    // 5. Caso general: lista no vacía
    list.last^.sig := newNode;
  list.last := newNode;  // 6. Actualiza el puntero al último nodo
end;

```

Veamos un ejemplo. Partimos del estado inicial:

```

[10] -> [20] -> nil
first↑   last↑

```

Ejecutamos la instrucción de inserción: `insert_at_end(list, 30)`

Sucede lo siguiente:

1. Crear nuevo nodo: `[30] -> nil`
2. Actualizar enlace: `[20] -> [30]`
3. Actualizar last: last ahora apunta a `[30]`

```

Estado final: [10] -> [20] -> [30] -> nil
               first↑           last↑

```

En cuanto a la **inserción al principio** de la lista, se trata de una operación particularmente eficiente y útil en aplicaciones donde se necesita acceso rápido a los elementos más recientes. Su complejidad es $O(1)$:

```

procedure insert_at_begin(var list: TListaSimple; x: integer);
var
  newNode: PNode;
begin
  new(newNode);
  newNode^.info := x;
  newNode^.sig := list.first; // El sig. al nuevo apunta al antiguo primero
  list.first := newNode;     // Actualizar puntero al primer nodo
  if list.last = nil then    // Si la lista estaba vacía
    list.last := newNode;    // El nuevo nodo también es el último
end;

```

Eliminación de elementos

Para eliminar tenemos ambas opciones, igual que en la inserción. La **inserción al principio de la lista** es una operación eficiente: Complejidad $O(1)$:

```

procedure delete_at_begin(var list: TListaSimple);
var
  temp: PNode;
begin
  if not is_empty(list) then
  begin
    temp := list.first;           // Guardar referencia al nodo a eliminar

```

```

        list.first := list.first^.sig; // Actualizar el primer puntero
        dispose(temp);                // Liberar memoria
        if list.first = nil then      // Si la lista quedó vacía
            list.last := nil;         // Actualizar puntero al último
        end;
    end;
end;

```

La otra operación de **inserción al final** requiere recorrer toda la lista para encontrar el penúltimo nodo. Su ineficiencia relativa justifica el uso de listas doblemente enlazadas cuando se necesitan eliminaciones frecuentes al final:

```

procedure delete_at_end(var list: TListaSimple);
var
    current, prev: PNode;
begin
    if not is_empty(list) then
        begin
            current := list.first;
            prev := nil;
            // Recorrer hasta el último nodo
            while current^.sig <> nil do begin
                prev := current;
                current := current^.sig;
            end;
            if prev = nil then begin // Solo había un elemento
                dispose(list.first);
                list.first := nil;
                list.last := nil;
            end
            else begin // Había más de un elemento
                dispose(current);
                prev^.sig := nil;
                list.last := prev;
            end;
        end;
    end;
end;

```

Pertenencia de un elemento

La función **in** determina si un elemento está o no en la lista. A continuación proponemos dos implementaciones, una iterativa y otra recursiva:

```

function in_list(list: TListaSimple; x: integer): boolean;
var
    current: PNode;
begin
    current := list.first;
    // Recorremos la lista hasta encontrar el elemento o llegar al final
    while (current <> nil) and (current^.info <> x) do
        current := current^.sig;
    end;
    in_list := current <> nil;
end;

function rec_in_list(list: TListaSimple; x: integer): boolean;
var sublist : tListaSimple;
begin
    if list.first = nil then rec_in_list := false // Caso base I: lista vacía
    else if list.first^.info = x then rec_in_list := true // Caso base II: está
    else begin // Llamada recursiva con una sublista sin el primer elemento
        sublist.first := list.first^.sig;
        sublist.last := list.last;
        rec_in_list := rec_in_list(sublist, x);
    end;
end;

```

En cuanto a su complejidad, el mejor escenario ocurre cuando el elemento buscado se encuentra en la primera posición, lo que permite resolver la operación en tiempo constante, $O(1)$. El peor caso aparece cuando el elemento está al final de la lista o no está en ella, situación en la que es necesario recorrerla por completo, alcanzando así un coste computacional de $O(n)$. Si consideramos una distribución uniforme de las posiciones posibles, el caso promedio implica revisar aproximadamente la mitad de la lista, lo que se aproxima a $O(n)$ pese a expresarse como $O(n/2)$.

En cuanto a la implementación recursiva, si bien resulta conceptualmente elegante, presenta algunas desventajas prácticas que conviene tener en cuenta: Cada llamada recursiva ocupa espacio en la pila, de modo que en el peor de los casos el consumo crecería linealmente con el tamaño de la lista. Esto puede convertirse en un problema en entornos donde la memoria de la pila es limitada o está estrictamente controlada, haciendo que este enfoque sea menos adecuado en situaciones de gran profundidad o en sistemas con recursos restringidos.

Otras operaciones

Otras operaciones que podemos incluir en nuestro TAD ListaSimple serían:

```
function to_string(list: TListaSimple): string;
// genera una cadena de texto con el contenido de los elementos de la lista
var
  current: PNode;
  str: string;
begin
  str := '';
  current := list.first;
  while current <> nil do begin
    str := str + IntToStr(current^.info) + ' ';
    current := current^.sig;
  end;
  to_string := str;
end;

function num_elems(list: TListaSimple): integer;
// Cuenta el número de elementos de la lista
var
  current: PNode;
  count: integer;
begin
  count := 0;
  current := list.first;
  while current <> nil do begin
    count := count + 1;
    current := current^.sig;
  end;
  num_elems := count;
end;

procedure copy(list: TListaSimple; var list2: TListaSimple);
// Genera una copia de la lista
var
  current: PNode;
begin
  initialize(list2);
  current := list.first;
  while current <> nil do begin
    insert_at_end(list2, current^.info);
    current := current^.sig;
  end;
end;
```

3. Lista circular

En una lista circular, el último nodo apunta al primero, formando un ciclo. Esto elimina la necesidad de manejar casos especiales para el final de la lista. Se gestiona mediante un único puntero, que apunta al último de la lista, lo que de manera ingeniosa permite acceder también fácilmente al primero de la lista: acceder al primero es tan sencillo como consultar el campo sig del último elemento, lo que simplifica considerablemente su manejo.

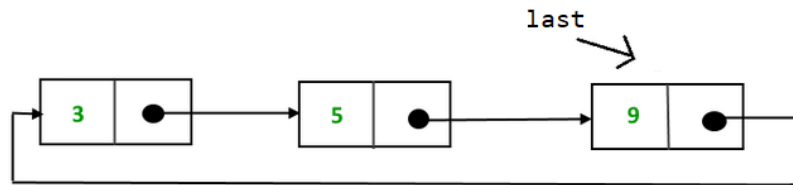


Ilustración 14. Lista circular

En este tipo de estructura no existen valores NIL, ya que todos los nodos tienen siempre un sucesor definido. Esto permite realizar un recorrido continuo, capaz de iterar sin detenerse mientras se siga avanzando por la secuencia. Su implementación en Pascal parte de la siguiente estructura de datos:

```
type
  tListaCircular = record
    last: PNode;
  end;
```

En cuanto a sus operaciones, algunas de ellas no sufren modificaciones con respecto a la lista no circular vista anteriormente, pero la mayoría sí ya que la estructura de manejo es distinta:

```
procedure initialize(var list: tListaCircular);
begin
  list.last := nil;
end;

function is_empty(list: tListaCircular): boolean;
begin
  is_empty := list.last = nil;
end;

function first(list: tListaCircular): integer;
begin
  if not is_empty(list) then first := list.last^.sig^.info;
end;

function in_list(list: tListaCircular; x: integer): boolean;
var
  aux: ^nodo;
  found: boolean;
begin
  found := false;
  if not is_empty(list) then begin
    aux := list.last^.sig;
    while (aux <> list.last) and not found do begin // busca el elemento
      if aux^.info = x then found := true
      else aux := aux^.sig;
    end;
    if aux^.info = x then found := true;
  end;
  in_list := found;
end;
```

```

procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    // Caso 1: Lista vacía (no hacemos nada)
    if not is_empty(list) then begin
        // Caso 2a: Un único nodo en la lista
        if list.last^.sig = list.last then begin
            if list.last^.info = x then begin
                // Si el único nodo es el que queremos eliminar
                begin
                    dispose(list.last);
                    list.last := nil;
                end;
            end;
        end
        // Caso 2b: Más de un nodo, buscamos el elemento
        found := false;
        prev := list.last; // El anterior al primero es el último
        current := list.last^.sig; // Empezar a buscar por el primero
        repeat
            if current^.info = x then // Encontrado el nodo a eliminar
                found := true
            else begin
                prev := current; // Avanzar puntero al nodo anterior
                current := current^.sig;
            end;
        until found or (current = list.last^.sig);

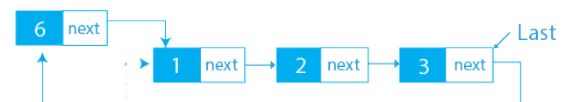
        // Si existe, lo eliminamos
        if found then begin
            // "Punteamos" el nodo actual
            prev^.sig := current^.sig;
            // Si borramos el último nodo, actualizamos last
            if current = list.last then
                list.last := prev;
            dispose(current);
        end;
    end;
end;
end;
end;

```

```

procedure insert_at_begin(var list: tListaCircular; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    if is_empty(list) then begin
        aux^.sig := aux;
        list.last := aux;
    end
    else begin
        aux^.sig := list.last^.sig;
        list.last^.sig := aux;
    end;
end;
end;

```



```

procedure insert_at_end(var lista: tListaCircular; x: integer);
var
  aux: PNode;
begin
  new(aux);
  aux^.info := x;
  if is_empty(lista) then
    aux^.sig := aux
  else begin
    aux^.sig := lista.last^.sig;
    lista.last^.sig := aux;
  end;
  lista.last := aux;
end;

```



```

procedure copy(list: tListaCircular; var list2: tListaCircular);
var
  aux, new_node: ^nodo;
begin
  initialize(list2);
  if not is_empty(list) then begin
    aux := list.last;
    repeat
      insert_at_end(list2, aux^.info);
      aux := aux^.sig;
    until aux = list.last;
  end;
end;

```

La siguiente tabla compara las dos implementaciones anteriores:

	Lista Simple	Lista Circular
Estructura	Tiene un inicio y un final bien definidos	Todos los nodos forman un ciclo cerrado
Punteros nulos	El último nodo tiene sig = nil	No existen punteros nulos
Recorrido completo	Termina cuando current = nil	Termina al volver al nodo inicial
Inserción al principio	O(1)	O(1)
Inserción al final	O(1) si se mantiene un puntero last	O(1)
Eliminación al final	O(n)	O(n)
Aplicaciones típicas	Procesamiento secuencial general	Procesamiento cíclico, buffers circulares, tareas repetitivas

Digresión: Comprendiendo y recordando los métodos

En este punto queremos que reflexiones. Has estudiado ya varias estructuras de datos: pilas, colas, listas, y pronto veremos más. Has visto muchos métodos, implementaciones que te hemos proporcionado y que debes ser capaz de recordar, pero... ¿Es la implementación que te mostramos aquí la única manera de hacer las cosas? La respuesta a esta pregunta es: rotundamente NO.

El código que te mostramos para cada método es una solución correcta y válida, pero no es la única. En programación, y especialmente trabajando con punteros, hay muchas formas de llegar al mismo resultado. Nuestras soluciones suelen ser muy académicas porque primamos la claridad ya que los materiales tienen un objetivo de aprendizaje, sobre todo.

Dicho esto, nos permitimos darte algunos consejos que esperamos te sean útiles:

- No memorices el código. Aprenderse de memoria líneas como esta del método delete: `prev^.sig := current^.sig` es inútil si no visualizas lo que ocurre.
- Entiende la lógica de punteros. Dibuja tu estructura de datos en un papel. Mueve las flechas (punteros). Entiende por qué necesitamos un puntero `prev` o por qué el `last` es especial.
- Haz tu propia implementación. Si entiendes el concepto (buscar → desenlazar → liberar), podrás escribir tu propio código sin necesidad de copiar el nuestro.

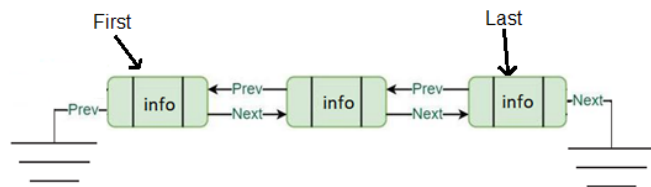
Recuerda: La memoria falla, la lógica permanece.

4. Listas doblemente enlazadas

Las listas doblemente enlazadas pueden entenderse como una evolución de las listas simples, ya que incorporan la posibilidad de desplazarse en ambos sentidos. Aquí, cada nodo incluye dos punteros: uno que apunta al siguiente elemento y otro que referencia al anterior, lo que permite recorrer la estructura de manera bidireccional. Gracias a esta organización, eliminar un nodo concreto resulta muy eficiente, pues basta con tener una referencia directa a él para eliminarlo en tiempo constante. Del mismo modo, insertar nuevos elementos antes o después de un nodo existente no requiere recorrer la lista, lo que hace a estas operaciones especialmente flexibles y rápidas. Su implementación sería:

```
type
  tNodoDoble = record
    info: integer;
    sig: ^tNodoDoble;
    ant: ^tNodoDoble;
  end;

  tListaDoble = record
    first: ^tNodoDoble;
    last: ^tNodoDoble;
  end;
```



Este tipo de estructura se emplea con frecuencia en aplicaciones donde el movimiento hacia adelante y hacia atrás es fundamental. Es el caso de los historiales de navegación en los navegadores web, los sistemas de edición de texto que permiten deshacer y rehacer acciones, o los mecanismos de caché como las implementaciones LRU (*Least Recently Used* o Menos Usada Recientemente). También constituyen la base de estructuras más complejas, como los deque o ciertos tipos de árboles, donde la eficiencia en la manipulación de nodos es especialmente valiosa.

Una implementación en Pascal de sus operaciones básicas sería la siguiente:

```
procedure initialize(var list: tListaDoble);
begin
  list.first := nil;
  list.last := nil;
end;

function is_empty(list: tListaDoble): boolean;
begin
  is_empty := list.first = nil;
end;

function first(list: tListaDoble): integer;
begin
  if not is_empty(list) then first := list.first^.info;
end;

function last(list: tListaDoble): integer;
begin
  if not is_empty(list) then last := list.last^.info;
end;
```

```

procedure insert_at_end(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := nil;
    aux^.ant := list.last;
    if list.last=nil then list.first := aux
    else list.last^.sig := aux;
    list.last := aux;
end;

procedure insert_at_begin(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := list.first;
    aux^.ant := nil;
    if list.first = nil then list.last := aux
    else list.first^.ant := aux;
    list.first := aux;
end;

procedure delete_at_end(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then begin
        aux := list.last;
        list.last := aux^.ant;
        if list.last <> nil then
            list.last^.sig := nil
        else
            list.first := nil;
        dispose(aux);
    end;
end;

procedure delete_at_begin(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then
        begin
            aux := list.first;
            list.first := aux^.sig;
            if list.first <> nil then
                list.first^.ant := nil
            else
                list.last := nil;
            dispose(aux);
        end;
end;

function in_list(list: tListaDoble; x: integer): boolean;
var
    aux: ^nodo;
    found: boolean;
begin
    aux := list.first;
    found := false;
    while (aux <> nil) and not found do

```

```

if aux^.info = x then found := true;
else aux := aux^.sig;
in_list := found;
end;

```

5. Aplicaciones y estructuras derivadas

La flexibilidad de las listas, tanto simples como doblemente enlazadas, permite utilizarlas como base para construir estructuras de datos más ricas y con comportamientos especializados. Su capacidad para gestionar nodos de manera dinámica, sin necesidad de mover grandes bloques de memoria, facilita adaptar su funcionamiento a distintos tipos de problemas.

Por ejemplo, una lista puede transformarse en una lista ordenada simplemente garantizando que cada elemento nuevo se inserte en la posición adecuada. Esto permite mantener los datos siempre ordenados sin recurrir a procesos de reordenación completos cada vez que se modifica la estructura.

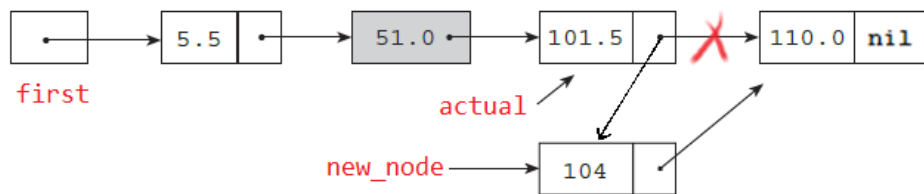


Ilustración 15. Inserción en una lista ordenada

El código de esta operación en Pascal sería:

```

procedure insert(var list: tSortedList; x: integer);
var
  new_node, actual: ^nodo;
begin
  new(new_node);
  new_node^.info := x;
  if is_empty(list) or (x < list.first^.info) then begin
    new_node^.next := list.first; // Insertar el primero
    list.first := new_node;
  end
  else begin // Situar actual en el nodo anterior a la inserción
    actual := list.first;
    while (actual^.next <> nil) and (x > actual^.next^.info) do
      actual := actual^.next;
    end;
    new_node^.next := actual^.next;
    actual^.next := new_node;
  end;
end;

```

Si, por el contrario, añadimos la idea de prioridad, la misma estructura puede servir para implementar colas de prioridades. En ellas, los elementos no se gestionan por orden de llegada, sino según un criterio que determina cuáles deben procesarse antes, algo muy útil en planificación de tareas o en algoritmos de búsqueda informada.

Las listas doblemente enlazadas también facilitan la creación de deque, estructuras en las que se puede insertar y eliminar información tanto por el inicio como por el final con la misma eficiencia. Este comportamiento resulta útil en áreas como la simulación, la gestión de buffers o ciertas estrategias de búsqueda.

Además, las listas pueden participar en la construcción de mapas o diccionarios, especialmente cuando necesitamos agrupar elementos que comparten un mismo valor hash. En estos casos, la lista actúa como contenedor flexible para almacenar pares clave-valor de manera organizada.

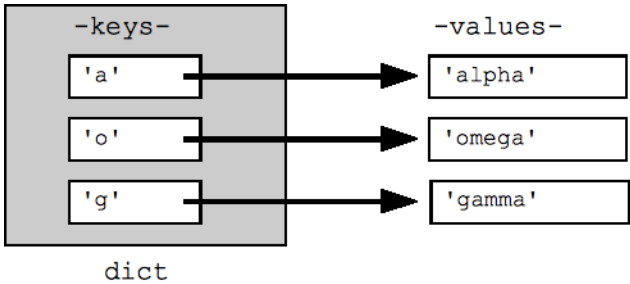


Ilustración 16. Mapa o diccionario

Por último, desempeñan un papel esencial en la representación de grafos. Cada nodo de un grafo puede mantener una lista con sus conexiones o adyacencias, lo que permite manejar estructuras complejas de relaciones tales como redes, rutas o dependencias, de una manera eficiente y relativamente sencilla de implementar.

En definitiva, las listas enlazadas funcionan como bloques fundamentales a partir de los cuales es posible desarrollar estructuras de datos más avanzadas. Comprender su funcionamiento y sus posibilidades abre la puerta a diseñar soluciones eficientes y adaptadas a cada problema.

La elección entre lista simple, circular, doble u ordenada depende de los requisitos específicos de operaciones y del patrón de uso esperado. Ser consciente de la complejidad de las operaciones, ayuda a tomar la decisión en cada caso. La siguiente tabla ofrece un resumen comparativo basado en la complejidad de las operaciones más habituales para cada estructura:

Operación	Lista Simple	Lista Circular	Lista Doble	Lista Ordenada
Insertar inicio	$O(1)$	$O(1)$	$O(1)$	$O(n)^*$
Insertar final	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Insertar medio	$O(n)$	$O(n)$	$O(1)^{**}$	$O(n)$
Buscar	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Eliminar inicio	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Eliminar final	$O(n)^\dagger$	$O(1)^\ddagger$	$O(1)$	$O(n)$
Recorrer	$O(n)$	$O(n)$	$O(n)$	$O(n)$

* Depende de si la lista mantiene el orden en la inserción.
 ** Requiere tener ya una referencia al nodo donde se inserta.
 † Con puntero “last” pasa a ser $O(1)$.
 ‡ Requiere ajustar correctamente los punteros.